

The Control Tower, Explained

CCAR-P Domain 1 · Solution Design and Architecture · 17 percent, about 11 of 63 questions

A plain-language companion to the Control Tower Brief. Eight parts, one per chapter group, each with the everyday picture first, the exhibit code decoded line by line in words, the trap, the law, the tell for stems, and the tie-back to the Domain 3 lab and consignment KF-2481. Written in the same voice as the Loading Dock walkthroughs in this repository.

Part	Title	Brief chapters	Objectives
1	The Brief, and Objective 1.1: Ask Before You Draw	0, 1	1.1
2	The Augmented LLM: The Cell Every Organism Is Built From	2	1.3a
3	The Five Workflows: Arrangements of Clerks, Paths Fixed in Code	3	1.3b
4	The Agent: The Loop That Is Your 50 Percent Row	4	1.3c
5	Decomposition: Plans That Grow When the World Talks Back	5	1.5
6	Multi-Agent Systems: When One Detective Isn't Enough	6	1.4
7	End-to-End and the Boardroom's Five Words	7, 8	1.2, 1.6
8	The Atlas, the Blueprint Book and the Memory Map	Parts Two, Three, Memory Map	all

The five laws of Domain 1

- The constraint picks the pattern, not the ambition.
- Workflows when the path is known; agents when it isn't.
- Every loop needs a budget and an exit.
- Decompose on discovery, not in advance.
- Value is a pillar with a number attached.

Reading order from the brief: Day A reads the drill and hand-types E1 and E7. Day B sits the Blueprint Book cold. Day C reads the Atlas, retakes every miss and finishes the Memory Map. This explainer is for Day A and Day C. Per the standing rule in this repository, the exhibits are explained, not executed: they live in the book, not as standalone files.

The Brief, and Objective 1.1: Ask Before You Draw

Chapter 0 and Chapter 1 of the Control Tower Brief. What Domain 1 is, what Meera asked for, and why nobody writes a prompt yet.

What Domain 1 is

Domain 1 is Solution Design and Architecture. It carries 17 percent of the exam, about 11 of the 63 questions. Where Domain 3 asked how the pieces connect, Domain 1 asks a question that comes before any piece exists: what should we build at all, and how big should it be?

Objective	In plain words	Weight in the brief
1.1 Business problem to Claude solution	Translate a vague ask into a workload, a constraint and a measure of better	6 of 40 questions
1.2 End-to-end architecture	Input, processing, output, and the feedback loop nobody draws	6 of 40
1.3 Pattern selection	Workflow, agent, or plain augmented LLM. Your 50 percent row from Foundations	8 of 40, soft
1.4 Multi-agent orchestration	When one agent is not enough, and who tells whom what	6 of 40
1.5 Decomposition	Fixed steps versus subtasks discovered as results come back. Your 75 percent row	8 of 40, soft
1.6 Value pillars and SLAs	Efficiency, cost, productivity, performance, transformation, each with a number	6 of 40

The brief marks 1.3 and 1.5 as soft spots because they scored 50 and 75 percent on the Foundations mock. They get the most pages, the hardest exhibits, and two extra questions each in the Blueprint Book.

The bridge before the blueprint

The picture. A city hires an engineer and says: we need to cross the river. The amateur starts drawing a suspension bridge, because suspension bridges are magnificent. The engineer first asks how many cross per day, carrying what, in what weather, and what happens when the bridge is closed. Sometimes the answer is a ferry. Sometimes a tunnel. Occasionally it really is the suspension bridge.

Domain 1 is the discipline of asking before drawing. The exam fails the candidate who draws the magnificent thing when the ferry met the constraint. Anthropic's own engineering guidance opens with the same idea: find the simplest solution possible, and add complexity only when it demonstrably improves outcomes.

Meera's ask

Monday morning, Pune. Meera Nair, Head of Operations, calls Arjun Bose into the glass room beside the operations floor. Below them, 40 dispatchers handle about 40,000 shipments a day. Exceptions arrive by email, portal ticket and phone: delays, damage, customs holds, refund demands. Meera's ask is one sentence: *I want an AI layer over this desk. The board approved budget on Friday. Tell me what we build.*

Somewhere in the pile is consignment KF-2481, Padma Deshmukh's shipment, three days late. It is the same consignment that ran through every Domain 3 experiment. Here it becomes the running example of Domain 1 as well, which is useful: you already know its facts.

Arjun's job this week is not to write a prompt. It is to answer four questions in order, and those four questions are exactly what Domain 1 grades:

#	Arjun must decide	Objective	Part of this explainer
1	What is the actual problem, and would an AI solution beat the current desk on a pillar Meera can defend to the board?	1.1, 1.6	Parts 1 and 7
2	Which shape of system: a workflow, an agent, or just an augmented LLM?	1.3	Parts 2, 3, 4
3	How does the work decompose: fixed steps, or subtasks discovered as tool results come back?	1.5	Part 5
4	If several agents, who orchestrates whom, and where do humans sit in the loop?	1.4, 1.2	Parts 6 and 7

Trap: the magnificent bridge. The exam's favourite Domain 1 wrong answer is the most capable architecture in the option list. A stem that says the steps are the same every time, or the desk must answer in two seconds, has already picked a simple workflow. Choosing the multi-agent system anyway is over-engineering, and it appears in at least a quarter of Domain 1 items. But beware the reflex in reverse: when the stem says the subtasks cannot be predicted, the agent is the right answer, and refusing it is a reversed trade-off.

LAW 1 · The constraint picks the pattern, not the ambition. Read the stem for the constraint: the known path, the latency budget, the unpredictable subtasks, the regulated data. Let it choose. An option more capable than the constraint demands is not better. On this exam it is wrong.

Objective 1.1: the doctor, not the pharmacist

The picture. Walk into a pharmacy and ask for the strongest medicine you have. A good pharmacist refuses: strongest against what? A doctor starts from the symptom, the history, the allergy list, and only then reaches for the shelf. Objective 1.1 is being the doctor. The business problem is the patient. Claude is the shelf, not the prescription.

Translating Meera's ask means rewriting it until it names three things: a **workload**, a **constraint**, and a **measure of better**. Watch the sentence sharpen:

Version	The ask	What is still missing
Meera's	An AI layer over the desk.	Everything. This is ambition, not a problem.
Better	Reduce time-to-resolution on shipment exceptions.	Volume, constraint, target.
Exam-grade	Triage about 1,200 exceptions a day into 6 categories with a draft reply, first touch under 5 minutes, refunds above Rs 10,000 always reaching a human.	Nothing. Pattern selection can begin.

Read the exam-grade version again and notice that it already whispers its own architecture. *Triage into known categories* is a router. *Draft reply* is a chained step. *Refunds above a threshold to a human* is a gate. A well-translated problem makes objective 1.3 almost mechanical. That is why 1.1 stems reward the option that clarifies before building.

Where Claude fits, and where it should not

Signal in the stem	Reading	Family if ignored
Judgement over unstructured text at volume: emails, tickets, documents	Claude-shaped work	
A deterministic rule already covers it: if carrier is X, reroute to Y	Plain code. An LLM adds cost and new failure modes	OE over-engineered
Ambiguity a human currently resolves by asking a colleague	Claude drafts, a human confirms. A gate, not a replacement	HL human loop skipped
Real-time inside a hard control loop, like sortation machinery	Not an LLM's seat. Latency and determinism rule it out	CM constraint missed

Everyday example for the second row: a shop already has a sign saying parcels over 30 kilograms go to the freight desk. You do not hire a consultant to decide, parcel by parcel, whether each one is heavy. The rule exists. Writing it as code costs nothing and never hallucinates.

The tell

1.1 stems hide the decision in the **verb**. *Classify into six known categories* is a different problem from *investigate and decide*. The first is a workflow. The second smells of an agent. Underline the verb before you look at a single option.

How this connects to Domain 3

Everything you ran in the lab sits downstream of this chapter. The router in E6 is the triage step named here. The gate in E3 is the human gate on refunds. The SOP retrieval in E4 and E5 is how the draft reply gets its facts. Domain 3 built the rooms. Domain 1 decides whether the building should exist and how many floors it needs.

The one thing to remember. Translate the ask into a workload, a constraint and a measure of better before choosing anything. The constraint picks the pattern. The verb in the stem tells you which pattern it picked.

The Augmented LLM: The Cell Every Organism Is Built From

Chapter 2, objective 1.3a. One model call that can retrieve, call tools and hold memory. Exhibit E1 decoded in plain words.

One clerk with three drawers. Picture one clerk at one desk with three drawers: a filing cabinet (retrieval), a telephone (tools), and a notebook (memory). The clerk decides for himself when to open which drawer. He is not a department and not a process chart. One competent clerk, augmented. Every pattern in this book is just this clerk, photocopied and arranged.

The taxonomy with three floors

Anthropic's taxonomy stacks three floors. The **augmented LLM** is one model call that can retrieve, call tools and hold memory. **Workflows** are several such calls orchestrated through *predefined code paths*. **Agents** are systems where the model *dynamically directs its own process and tool usage*. Hold on to the two italic phrases. They are the exam's dividing lines and they appear in stems almost word for word.

Floor	Who decides the next step	Everyday picture
Augmented LLM	Nobody has to. One question, one answer, maybe one phone call	A clerk answering one customer
Workflow	Your code, written before any model speaks	An assembly line the clerk stands in
Agent	The model, by looking at each result	A detective choosing the next lead

Exhibit E1: KAVI triages one exception

The book's first exhibit is about 40 lines. One call, one tool, no loop beyond a single round-trip. You do not need to run it. You need to be able to tell its story, line by line, because the agent loop in Part 4 is this same story with a for-loop wrapped around it.

Lines	What happens	In plain words
1 to 4	Imports. DRY_RUN read from the environment. MODEL set to a current alias.	The light switch by the door. Flip DRY_RUN on and the whole building runs on battery: no bill, no network. The free probe always comes before the paid call, the same habit as DRY_RUN in E2 and E5 of the lab.
6 to 12	tools: one entry named get_shipment_status, with a description and a JSON schema that requires consignment_id.	A tool is a promise, not code. It is a phone-book entry: here is my name, here is what I do, to call me give me a consignment id string. The required list says: no id, no call.
14 to 15	fake_status returns a canned record: CUSTOMS_HOLD, three days late.	The canned world. When the building is on battery, someone still has to answer the phone. Your prediction stays something solid to check against.
17 to 20	triage() opens. If DRY_RUN, return a canned verdict and stop. Otherwise create the client.	First move: on battery, hand back the canned verdict. The paid clerk is never disturbed until the free rehearsal has run.
21 to 28	One messages.create: system says reply as JSON in six categories, tools hands over the phone book, messages carries the email as the user turn.	The one paid call. The system prompt is the clerk's standing orders. The tools list is the phone book. The email rides in as the user turn.
29	if msg.stop_reason == tool_use	The hinge of the whole chapter. The response's stop_reason is the clerk's note on top of the file. tool_use means: I picked up the phone, connect me. Anything else means the file is finished.
30 to 32	Find the tool_use block, read its input, call fake_status with it.	Reading the phone request and dialling. In production this line is the real MCP tool from the lab's E1.
33 to 38	A second messages.create with the whole conversation so far replayed, plus one new user turn carrying the tool_result, matched by tool_use_id.	The receipt. The API is stateless, so you replay everything: the email, the clerk's reply, then the result. tool_use_id matches the receipt to the request the way a courier receipt matches a parcel. Forget to replay the assistant turn and the clerk has amnesia.
39 to 41	Return the text. Print the triage of Padma's email.	Padma's email is triaged. One exception, at most two paid calls, no loop. This is the cell.

The habit the book drills: predict before run

Before the battery run, write down what will print and how many paid calls were made. The answer is the canned verdict and zero. Before the live run, write down which branch fires. For Padma's email the model calls the tool, because the email names a consignment and asks where it is, and the clerk cannot know that without picking up the phone. The point is not the answer. The point is having written a prediction that the run can prove wrong.

Sabotage lab: break exactly one variable

Delete the required list from the schema. Now the model may call the tool with an empty input, and fake_status throws a KeyError. The lesson, in the book's words: **the schema is a guardrail you own; the model's compliance is a behaviour you rent**. Put the line back. One variable, one lesson.

Drift: prefilled assistant turns. Older material forces JSON by pre-filling the assistant's reply with an opening brace. Current docs: assistant-message prefill is unsupported on newer models and returns a 400. The current route is structured outputs through output_config.format with a JSON schema, or instructions plus validation in code, as E1 does. I checked this against the

API reference loaded earlier in this session, and it agrees: prefill is removed on the current model family.

How this connects to Domain 3

The clerk's telephone is the MCP tool the lab's E1 served. The phone book is the schema E2 priced at about 193 tokens each. The receipt matched by tool_use_id is the same round-trip the lab's probe made when it called get_shipment_status for KF-2481. Domain 3 showed the phone network. Domain 1 shows the clerk deciding whether to pick up the phone.

The one thing to remember. *The augmented LLM is one call that can retrieve, call tools and hold memory. Its reply carries a stop_reason, and that one field decides whether there is a next move. Everything bigger is this cell, arranged.*

The Five Workflows: Arrangements of Clerks, Paths Fixed in Code

Chapter 3, objective 1.3b. Chaining, routing, parallelization, orchestrator-workers and evaluator-optimizer. Exhibits E2, E3, E4 and E6 decoded.

A **workflow** is several augmented-LLM calls whose sequence your code decides before any model speaks. The model fills in steps. It never chooses them. Five named arrangements cover the exam, and the book says to learn each as a chain of four: shape, when, Kaveri scene, trap.

3.1 Prompt chaining (E2): the assembly line

The picture. Three clerks in a row. The first sorts the letter, the second drafts the reply, and between them stands a supervisor with a checklist who bins anything malformed. Nobody improvises the order. Accuracy goes up. Latency goes up too, because you queued three desks.

When: the task decomposes into fixed subtasks known in advance, and you will trade latency for accuracy. The programmatic gate between steps is the pattern's signature.

Lines	What E2 does	In plain words
1	VALID is a set of six category words.	The supervisor's laminated checklist. Six words, nothing else passes.
3 to 5	classify() asks a small model for one word, the category only.	Desk one. A small cheap model is enough to say customs. Each desk is exactly as smart as its step needs.
7 to 10	gate() raises ValueError if the category is not in VALID.	The gate is code, not model. If desk one mumbles something customs-ish, the line stops instead of desk two writing a confident reply to a nonsense category. Deterministic checks between steps are the whole reason chaining beats one big prompt.
12 to 15	draft_reply() asks the main model for a courteous reply under 120 words with no compensation promises.	Desk two writes to Padma. It is told the category and the boundaries. It never re-decides the category. The line has one direction.
17 to 18	reply = draft_reply(EMAIL, gate(classify(EMAIL)))	The whole line in one expression: classify, gate, draft. Your code owns this order forever. When a stem says the steps are the same every time, this, not an agent, is the answer.

Trap: a chain where a lookup would do. If the category is already a field on the ticket, desk one is an LLM re-deriving what a database column knows. Over-engineering wears a lab coat here: it looks diligent. Check the stem for data that already exists before you spend a model call creating it.

3.2 Routing (E3): the hospital triage nurse

The picture. One nurse at the door, thirty seconds per patient, sending each to the right specialist ward. The nurse treats nobody. A cheap decision at the front, expensive expertise only where it is needed.

When: distinct input categories exist, each better served by a different prompt or model, and classification is reliable. Routing separates concerns: improving the refund specialist cannot break the damage specialist.

Lines	What E3 does	In plain words
1 to 8	SPECIALISTS maps each category to a model and a system prompt: refund to the large model, customs to the mid model, delay to the small model.	Three door signs. Read the model column with Domain 2 eyes: the refund ward gets the big model because money and judgement live there; delays get the small one because an apology has a template's heart. Routing is where model selection stops being one decision and becomes a decision per category.
10 to 13	route() reuses E2's classify and gate, then looks up the specialist with .get(cat, default delay) and makes one call.	The nurse is E2's classifier, reused. One new idea: .get with a default. An unmapped category falls to a safe ward instead of crashing at two in the morning. Cheap insurance against a silent failure.
15	One email in, one specialist out.	The path was chosen by code reading one word. The model never saw a menu of wards. That is what keeps this a workflow, not an agent.

Trap: the router grading its own homework. A stem offering to let the specialist model also verify the routing was correct is inviting a circle: the expensive call re-doing the cheap call's job. If routing accuracy worries you, the answer lives in Domain 4, an evaluation set for the classifier, not in doubling the pipeline.

3.3 Parallelization (E4): three scouts, or three judges

The picture. Send three scouts down three different roads at once. The answer arrives when the slowest scout returns. That is sectioning. Ask three judges the same question and take the majority. That is voting. Same trick, running things at the same time, for opposite purposes: coverage versus confidence.

Lines	What E4 does	In plain words
1 to 5	check(carrier, cid) is an async function asking the small model for one ETA line.	One scout, one road. async means a scout who does not make the others wait at the gate while he walks.
7 to 10	sectioning() gathers check() for BlueDart, Delhivery and IndiaPost at once.	asyncio.gather releases all three together. Wall-clock time is the slowest scout, not the sum: three two-second calls take about two seconds instead of six. When a stem waves a latency budget at independent subtasks, this is the tell.
12 to 17	voting() asks the main model APPROVE or REJECT three times and takes the majority.	Same claim, three independent verdicts, majority rules. Cost triples for one decision, so voting is spent where being wrong is expensive, like refunds, not where it is cheap, like apologies.
19 to 20	Both halves run.	Notice what did not happen: no model chose to parallelize. Your code drew the fork. Workflow.

Trap: voting on facts. Voting raises confidence on judgement. It cannot vote a fact into existence. Three models guessing an ETA will agree on a hallucination as happily as on the truth. The scout for a fact is a tool call, not a ballot. Right mechanism, wrong axis: the reversed trade-off family.

Domain 3 connection: the carrier ETA call in One Ticket ran concurrently with the tail of retrieval and came back in 610 ms with a 3 second timeout. That was sectioning with a budget, the same shape as this exhibit.

3.4 Orchestrator-workers (E5): the site foreman

The picture. A foreman walks the site each morning and writes that day's task list himself. Nobody handed him one. Masons, electricians and plumbers each get a chit. The foreman reads their reports and synthesises. The list is different every morning because the site is.

When: subtasks cannot be predicted in advance, so the model must decompose. This is the boundary stone between workflows and agents. The *structure*, delegate then work then synthesise, is still fixed code. The *content* of the plan is the model's. E5's code and its line-by-line decode live in Part 5, where objective 1.5 needs them.

3.5 Evaluator-optimizer (E6): the writer and the editor

The picture. One clerk drafts. A second, colder clerk marks it against a rubric and hands it back. Two rounds of this and the letter is noticeably better. But without a page limit the pair would polish forever, so the editor's desk has a rule: two revisions, then it ships.

Lines	What E6 does	In plain words
1 to 2	draft_and_polish(email, max_rounds=2). First draft is free-form.	The letter to Padma gets an editor. Everything after this answers to the rubric.
3	for round_no in range(max_rounds)	The budget: at most two rounds. Law 3 in four characters. An unbounded while-loop here is the classic silent way to spend Rs 400 polishing a delay apology.
4 to 6	The evaluator gets the rubric (tone courteous? facts only? under 120 words?) and the draft. It replies PASS or one specific criticism.	The editor sees the rubric and the draft, nothing else. Clear criteria are the pattern's entry fee. No rubric, no evaluator, just two models exchanging vibes.
7 to 8	if verdict == PASS: break	The exit. PASS breaks early. Budget and exit: E7's agent loop will need exactly the same two organs, grown larger.
9 to 11	Revise with the criticism and the draft. Return the last draft.	Targeted feedback on a specific failure, which is precisely the iterative-refinement discipline the 50 percent row was testing. The letter that ships is the last draft, on PASS or when the budget runs out.

Trap: evaluator without criteria. Have a second model review it, with no rubric, is the sounds-diligent ghost in a new coat. It measures nothing. If the stem names no criteria, the evaluator option is decoration and the correct answer usually fixes the first prompt instead.

All five on one card

Pattern	Everyday picture	Signature in code	When	Trap
Chaining	assembly line with a supervisor	a gate function between steps	fixed steps, accuracy over latency	chaining what a lookup knows
Routing	triage nurse	a dict of specialists, one call	known categories, different handling	router grading its own homework
Parallel	three scouts or three judges	asyncio.gather	independent subtasks, or 3x cost for confidence	voting on facts
Orchestrator-workers	site foreman	model writes the plan, code steers	subtasks unpredictable	decomposing the known
Evaluator-optimizer	writer and editor	rubric, max_rounds, PASS	a rubric exists, drafts improve	evaluator without criteria

LAW 2 · Workflows when the path is known; agents when it isn't. All five arrangements share one property: code owns the path. The moment no path can be fixed in advance, and the model must look at each result to choose its next act, you have left this chapter.

The one thing to remember. Five arrangements of one clerk. Code picks the order every time. Learn each as shape, when, scene, trap, and you can recognise a diagram described in words.

The Agent: The Loop That Is Your 50 Percent Row

Chapter 4, objective 1.3c. Exhibit E7 decoded branch by branch, the seven stop_reason values, and the loop that never asks why it stopped.

The detective on a retainer. A workflow clerk follows a checklist. A detective is hired differently: find out why KF-2481 is stuck, here are your tools, here is your expense cap, report when done. Each clue decides the next move. Nobody scripted the investigation. The retainer has two clauses every stem cares about: the expense cap (budget) and report when done (exit). A detective without both is not employed. He is loose.

An **agent** is a model dynamically directing its own process and tool usage. Mechanically it is one thing only: **a loop around messages.create that keeps going while the model asks for tools and stops when it doesn't**. The field that steers the loop is stop_reason. The Foundations report says this mechanism cost you half your marks, so the book gives it the slowest treatment, and so does this part.

Exhibit E7, branch by branch

Lines	What E7 does	In plain words
1 to 3	MODEL set. BUDGET = 6. DRY_RUN read.	The retainer is signed. The expense cap is written in line two, before the loop exists. Law 3 is written before the loop, never as an afterthought at the bottom.
5 to 10	run_tool(call) handles the two known tools and returns a structured error for anything else.	The toolbag. Same promise shape as E1's phone book, now two tools. Line 10 matters: an unknown tool name returns an error result, not an exception. The detective learns his tool failed and can adapt. Crashing the loop would teach him nothing.
12 to 14	msgs starts with one user turn, the brief. Client is None on battery.	The case file opens with the brief. Everything the investigation learns is appended here, because the API is stateless. The file is the memory.
15	for turn in range(1, BUDGET + 1)	The loop is bounded by construction. The exam's favourite wrong option runs while True with the budget checked somewhere else. Somewhere else is nowhere.
16 to 17	On battery: print the turn and return a dry-run verdict.	Why exactly one line prints under DRY_RUN: the first turn returns before any paid call.
18 to 19	One messages.create per turn with tools and the whole case file.	One paid call per turn. The entire file goes up, one decision comes back. The response's stop_reason is the detective's note clipped to the front.
20	match r.stop_reason	The match statement is the whole objective. Every branch below is one value of that one field.
21 to 26	tool_use: append the assistant content, run every tool he asked for, append a user turn with all the tool_result blocks.	I need the phone. Three moves in order: file his request, run every tool, file the receipts matched by tool_use_id. Miss the first and the API rejects the receipts as orphans. The loop then comes round again, because the model's own reply decides whether there is a next turn. That sentence is your 50 percent row, verbatim.
27 to 28	end_turn: return the text.	Case closed, here is my report. The natural exit. Do not loop again out of politeness; a loop that calls the API once more after end_turn is paying for a conversation with someone who already left.
29 to 31	max_tokens: append the content, append a user turn saying Continue.	The report hit the page limit mid-sentence. Truncated is not finished. File what arrived and ask for the rest. Treating max_tokens as success ships half a verdict, the silent failure's favourite door.
32 to 33	pause_turn: append the content and continue.	A server-side tool loop pausing for breath. Return the content as-is to resume. A newer value; most workbooks do not know it exists.
34 to 35	refusal: escalate with the reason model refused.	The detective declines the case. The wrong reflex is to retry automatically. The right one is a different path, here a human.
36 to 37	The wildcard: escalate with the unhandled value.	stop_sequence, model_context_window_exceeded, and whatever ships next quarter all land in escalation instead of an unhandled crash. A total match is what the phrase orchestration-layer safeguard looks like at line level.
38	After the loop: escalate(brief, budget exhausted).	The floor. Budget spent, case not closed. Every session ends in one of two ways: a completed resolution or a human hand-off with a reason. Never a silent stop.
40 to 43	escalate() returns a line naming Meera's desk, the reason and the brief.	Escalation is a first-class outcome, not an error string. Meera's desk learns why the machine gave up. That is a Domain 6 hand-off package in miniature.

Everyday example for stop_reason

Think of how a phone call ends. *Call me back after you check that* is tool_use. *That is all, goodbye* is end_turn. The line cutting out mid-sentence is max_tokens. *Hold please* is pause_turn. *I will not discuss that* is refusal. You would never treat a cut-off line as the end of the conversation. The loop must not either.

Drift: how many stop_reason values exist. Most study material teaches four stop_reason values: end_turn, max_tokens, stop_sequence, tool_use. The brief's reading of current docs lists seven, adding pause_turn (a server-tool loop paused, default limit 10 iterations, send content back to continue), refusal (declined; read stop_details), and model_context_window_exceeded (treat as truncated). All arrive as HTTP 200. A stop_reason is a status, never an error. The API reference loaded earlier in this session confirms the first six and that refusal carries stop_details; carry the seventh as the brief states it.

Trap: the loop that never asks why it stopped. Distractors love code that checks for tool_use and treats everything else as the finished answer. max_tokens then ships truncations; refusal ships an apology as a verdict. The tell in such stems is an **else that swallows**. This is the silent-failure family grown up, and at Professional level it is also a diagnosis question in Domain 4.

LAW 3 · Every loop needs a budget and an exit. E6 had max_rounds and PASS. E7 has BUDGET and end_turn, plus escalation as the floor beneath both. Two organs, every loop, no exceptions. When a stem's code lacks one, that absence is the answer.

Workflow, agent, or augmented LLM: the 1.3 decision in one table

The stem says	Choose	Because
Steps identical every time; accuracy over latency	Prompt chaining	fixed path plus gates
Known categories, different handling each	Routing	cheap sort, specialist wards
Independent subtasks plus a latency budget	Parallel (sectioning)	slowest-scout wall clock
High-stakes judgement, tolerance for 3x cost	Parallel (voting)	confidence by majority
Subtasks unpredictable, but delegate-then-synthesise is fixed	Orchestrator-workers	model plans, code steers
Clear rubric, drafts improve measurably	Evaluator-optimizer	criteria exist
Path itself unknowable; each result steers the next act	Agent	model directs, with budget and exit
One question, tools at hand, no multi-step anything	Augmented LLM	the cell suffices

Domain 3 connection: the lab's E6 router is a 1.3 routing workflow. The One Ticket walk, where every hop was fixed in advance, is a workflow end to end. The only agent-shaped moment in the whole Domain 3 series was the hypothetical tangled case sent to the agent tier with a budget.

The one thing to remember. An agent is a bounded loop around one call, steered by stop_reason, with a budget, an exit and an escalation floor. The model's own reply decides whether there is a next turn. Handle every value of that field, or the one you ignore becomes the failure you ship.

Decomposition: Plans That Grow When the World Talks Back

Chapter 5, objective 1.5. Static versus dynamic decomposition. Exhibits E5 and E8 decoded, and the growth cap that separates a planner from a spiral.

Packing versus walking. You can pack for a Himalayan trek in advance. The list is knowable. You cannot walk it in advance: at the river crossing you learn the bridge is out and the plan grows a detour. Static decomposition is packing: fixed subtasks, E2's assembly line. Dynamic decomposition is walking: subtasks appear because a result revealed them. Your 75 percent row was exactly this distinction, a plan that adapts versus a sequence that executes regardless.

Exhibit E5: the foreman writes the plan

Lines	What E5 does	In plain words
1 to 5	orchestrate(brief): plan = ask_json(main model, brief plus list 2 to 5 subtasks as JSON with id, task, needs).	KF-2481's customer is threatening to leave, fix it. The brief goes to the foreman, whose first paid act is writing the task list. Compare E2, where VALID and the function order were typed by Arjun. Here the subtasks are model-made and different for every brief. This single line is the boundary between 1.3's workflows and 1.5's decomposition.
6 to 10	findings = {}. For each subtask, gather the findings it needs, then ask the small model to do the subtask with those known facts.	The site book. Each subtask runs with its declared dependencies injected and nothing else. A worker checking can we re-route reads the status finding first. Workers are small models: the foreman thinks, the crew executes, the payroll stays sane.
11 to 13	Return ask(main model, brief plus all findings plus synthesise one recommendation for Meera).	The foreman reads every chit and writes one recommendation. Not a list of findings. A decision. Independent subtasks here could be E4's gather; patterns compose.

Exhibit E8: re-planning, the bridge is out

E5's plan, once written, still executes fixedly. E8 adds the trek's truth: a finding can change the plan. This is dynamic decomposition complete, and the book says to type it slowest.

Lines	What E8 does	In plain words
1 to 3	replan(brief, max_new=3). tasks = plan_from(brief). findings = {}, grew = 0.	The trek begins with E5's packed list, and a second budget: max_new = 3. A plan that can grow needs a cap on growth, or re-planning becomes the unbounded loop wearing a planner's clothing. Law 3 applies to plans, not just turns.
4 to 7	while tasks: pop the front task, run the worker with findings so far, file the result.	Walk one step. Take the front task, run it, file the finding. So far this is E5 with a queue.
8 to 12	verdict = ask_json(main model, given this new finding, does the remaining plan still hold? Reply ok true or false, add new subtasks, drop ids), sent with only the finding and the remaining tasks.	The walk-and-look. After every finding, one focused question: does the rest of the plan survive this? The model sees the new finding and the remaining tasks, not the whole history. Cheap, specific, structured. This call is what subtasks that adapt as new information is discovered means in running code.
13 to 17	if not ok and grew under max_new: drop the listed tasks, add the new ones, grew += number added.	The bridge is out. Customs hold discovered, so the re-route quote (pointless until release) is dropped, and file e-way bill and notify Padma appear. grew counts the detours against the cap. The plan changed because a result said so. Your 75 percent row, closed.
18	Return synthesise(brief, findings).	Same ending as E5. Decomposition patterns end the way they must: one recommendation, traceable to findings.

The expected trace, in the canned world

```
# run 1: status check -> CUSTOMS_HOLD, missing e-way bill
# replan: ok=false drop=[3] add=["file e-way bill", "notify Padma of ETA"]
# run 2: file e-way bill -> filed, release ~24h
# run 3: notify Padma -> draft ready
# grew=2, budget max_new=3 not exhausted
RECOMMENDATION: file the e-way bill today; hold the re-route (costlier
than the 24h release); send Padma the revised ETA with a goodwill note.
```

Read it against the Atlas A6 tree: subtasks 1 and 2 were planned (solid boxes), subtask 3 was planned and died the moment the hold was found (crossed out), subtasks 4 and 5 appeared because a finding said so (dashed boxes). If that tree could have been drawn before the run, chaining would have sufficed and orchestrator-workers was over-engineering. The dashed boxes are the exam's evidence that dynamic machinery earned its keep.

Sabotage lab

Change the growth check to always true and seed the replanner to always return ok false with one new subtask. Count the turns before you kill it. Lesson: **the growth cap is the difference between a planner and a spiral.** Fix it back.

LAW 4 · Decompose on discovery, not in advance. When subtasks are knowable, decompose in code (chaining), cheapest and most testable. When they aren't, let the model write the plan (orchestrator-workers). When findings can invalidate the plan, re-plan on findings with a growth cap. Escalating uncertainty, escalating machinery, never more than the stem demands.

Uncertainty level	Machinery	Exhibit	Budget organ
Subtasks knowable before the run	chain in code	E2	the fixed list is the budget
Subtasks unknowable, plan writable once	orchestrator-workers	E5	2 to 5 subtasks, declared needs
Findings can invalidate the plan mid-run	re-planning	E8	max_new growth cap

Trap: decomposing the known. A stem whose subtasks are listed in the stem itself does not need orchestrator-workers. The decomposition already exists; paying a model to rediscover it is over-engineering with extra steps. Dynamic machinery is for uncertainty that survives reading the brief twice.

Domain 3 connection: the worker that runs status check is calling the lab's E1 tool through the E3 gate. The worker that files the e-way bill would read SOP 214 section 4 through the E4 and E5 retrieval. Decomposition decides which workers exist. Domain 3 decides how each worker reaches its facts.

***The one thing to remember.** Three levels of uncertainty, three levels of machinery. Chain what you can list. Let the model plan what you cannot. Re-plan on findings only when findings can really change the plan, and cap the growth. The dashed boxes in the tree are your proof.*

Multi-Agent Systems: When One Detective Isn't Enough

Chapter 6, objective 1.4. The four properties, the three topologies, the caps to quote, and the coordinator-over-the-coordinator trap.

The agency, not the lone detective. Some cases outgrow one detective. The customs specialist, the carrier negotiator and the customer handler each need different tools, different briefs, different clearances. An agency has one partner who assigns cases and reads reports, and a strict rule: field notes stay in the field office; only the report crosses the partner's desk. That rule is context isolation, and it is why agencies scale where lone detectives drown.

Multi-agent means agents delegating to agents. In the Agent SDK, subagents are separate agent instances the orchestrator spawns, each running in its own conversation. Four properties carry the exam weight.

Property	Mechanism	Why the exam cares
Context isolation	A subagent explores in its own window; only its final message returns to the parent	A research subagent can read 40 documents without flooding the orchestrator. Domain 2's token economics, solved architecturally
Parallelism	Independent subagents run concurrently	Three reviews finish in the slowest one's time. E4's logic at agent scale
Specialised prompts	Each subagent gets its own system prompt and expertise	The customs agent's SOP knowledge would be noise in the negotiator's context
Tool restriction	Each subagent gets only its own tools; an omitted tool does not exist for it	The customer-facing agent physically cannot call process_refund. Configuration beats instruction, again

The delegation brief is everything. A subagent starts **fresh**, with no parent history, so the orchestrator's prompt to it must carry every path, id and decision it needs. Structured, complete handoffs between agent steps were your other 100 percent row at Foundations. At Professional they are load-bearing.

Everyday example: a hospital shift handover. The night doctor cannot say the patient in bed four, you know the one. The day doctor was not there. The handover sheet must carry the name, the history, the allergies and what was decided at 3 a.m., or the day shift starts from zero.

Choosing a topology

Topology	Shape	Reach for it when	It hurts when
Single agent	one loop, all tools	tools are a handful, one expertise	context bloats; one prompt serves three masters
Orchestrator plus subagents	partner assigns, field offices work, only reports travel	distinct expertises, parallelisable, isolation is valuable	subtasks so entangled every report needs every other's field notes; the isolation you bought becomes re-briefing overhead
Peer handoff	agent A completes, passes the baton to B	sequential phases with a clean break: triage shift then resolution shift	the baton is informal; an under-specified handoff drops context and authorization state

Everyday picture of the three: a general practitioner working alone (single agent); a GP who refers to specialists and reads their letters (orchestrator plus subagents); a hospital day shift handing over to the night shift (peer handoff).

Drift: guardrails around subagent growth. Common teaching: subagents can nest and parallelise, with no numbers attached. The brief's reading of current SDK docs names hard caps you set: spawn depth (default 3 layers, 1 means no nesting), concurrency (default 20 at once), and a per-query spend cap (max_budget_usd) that refuses new spawns, stops background subagents and ends the query at the limit. That is Law 3 at fleet scale. The spawning tool now appears as Agent in tool_use blocks; older versions said Task. These are Agent SDK facts from the brief; the Messages API reference I loaded does not cover them, so quote them as the brief gives them.

Trap: a coordinator over the coordinator. Adding a subagent is often correct. Adding a second orchestrator to manage the orchestrator almost never is. When two options both delegate, prefer the one with fewer layers and the same isolation. And remember the places where the agent or subagent genuinely is the answer: unpredictable subtasks, isolation, parallel expertise.

Domain 3 connection

Tool restriction is the lab's E3 gate and E2 per-route belt seen from the agent's side. The Wrong Tower flaw where permissions lived in the system prompt is the same lesson: never process refunds as an instruction is persuasion; omitting the tool from the subagent's list is physics. Context isolation is why the One Ticket window stayed at about 5,800 tokens: the retrieval happened somewhere that was not the drafting model's context.

The one thing to remember. Isolation, parallelism, specialised prompts, tool restriction. A subagent starts fresh, so the brief carries everything. Caps, not committees: depth, concurrency and spend. A second orchestrator is almost always the wrong answer.

End-to-End and the Boardroom's Five Words

Chapter 7 (objective 1.2) and Chapter 8 (objective 1.6). Input, processing, output, the loop nobody draws, and value as a pillar with a number attached.

Objective 1.2: the restaurant, not the recipe

The picture. A recipe is processing. A restaurant is end-to-end: the door (who gets in, in what state), the kitchen, the pass where the head chef tastes before plates leave, and the part amateurs forget, the comment cards that change next month's menu. An architecture diagram without the comment cards is a recipe pretending to be a restaurant.

Stage	Kaveri concretely	The decision that lives here
Input	Email, portal and phone-transcript exceptions normalised into one ticket schema	Validation and PII scrubbing happen here, before any model, not mid-pipeline
Processing	Router (E3) to specialist chains (E2), agent tier (E7) for the tangled cases	Patterns compose; each tier is as simple as its slice allows
Output	Draft replies, re-route orders, refund recommendations	The human gate: refunds at or above Rs 10,000 pause for Meera's desk. Domain 5 will formalise it
Feedback	Dispatcher edits diffed against drafts; outcomes logged; the eval set grows weekly (Domain 4)	Without this loop the system is frozen at launch quality forever

The Atlas A3 diagram draws the four stages as bands with a purple arrow running from feedback back up to processing. The brief calls that arrow the answer to a whole family of stems: when a 1.2 option list contains add a feedback loop from dispatcher edits, check whether the described architecture already has one. If not, that option beats every clever processing upgrade on the page.

The tell

1.2 stems love an architecture that is complete except for one absent arc, usually feedback or the human gate. The right option is rarely more processing. It is the **missing arc**. Count the four stages against every option before judging any of them clever.

Everyday example: a new cafe opens with a good menu and a good kitchen. Six months later it serves the same food to half the customers. Nobody read the comment cards. The kitchen was never the problem.

Objective 1.6: value pillars, the boardroom's five words

The household budget argument. We should get a dishwasher convinces nobody. We hand-wash 90 minutes a day; a Rs 40,000 machine returns that time inside five months ends the argument. Same purchase, but the second sentence names a pillar (time), a number (90 minutes) and a horizon (five months). Meera's board hears dishwasher arguments all day. Only numbered pillars survive.

The exam's five pillars, applied to the control tower. A solution is *aligned* when its design choices trace to a pillar with a number:

Pillar	Kaveri number	The design choice it justifies
Efficiency	First touch from 4 hours to under 5 minutes	Routing plus chaining tier for the routine 80 percent
Cost	Model spend at most Rs 6 per exception on average	Small models at the door (E3); big ones only where money moves
Productivity	Dispatchers at 3x throughput, editing not drafting	Draft-plus-gate output design, not auto-send
Performance / SLA	99 percent of drafts within 30 seconds	Parallel carrier checks; agent tier reserved for the tangled few
Transformation	Customs-hold pattern reports become a new pre-clearance service line	The feedback log is a product, not exhaust

Trap: the pillar swap. The reversed trade-off's home ground. The stem fixes one pillar, a 2 second SLA, a cost ceiling, and one option optimises a different, worthier-sounding pillar: maximise accuracy with a 3-model voting ensemble. Wrong. The stated pillar is the contract; the exam pays no marks for nobler physics than the customer ordered.

LAW 5 · Value is a pillar with a number attached. Improves customer experience is a wish. Cuts first-touch from 4 hours to 5 minutes at Rs 6 or less per exception is an architecture requirement, and every Domain 1 option can be tested against it. In 1.6 stems, the option that names measure, number and horizon beats the one that names technology.

Atlas A7: the value-pillar matrix

Boardroom Tuesday. Three candidate architectures scored the only way a 1.6 stem accepts, numerically per pillar. Green meets Meera's target; red breaches it.

Pillar and target	A: one big agent, every ticket	B: tiered, route + chains, agent for 20%	C: rules only, no LLM
Efficiency, first touch at most 5 min	about 2 min, meets	about 1 min, meets	4 h, breaches
Cost, at most Rs 6 per exception	about Rs 22, breaches (agent loops on trivia)	about Rs 4.10, meets	about Rs 0, meets
Productivity, 3x throughput	about 2.8x, meets	about 3.1x, meets	1x, breaches
SLA, 99% of drafts in 30 s	about 91%, breaches (loop latency tail)	about 99.3%, meets	no drafts
Transformation, new service lines	logs exist, unmined	customs pre-clearance line from the feedback log	none

Option B wins not because it is cleverest but because it is the only column with no red cell. **Alignment means no pillar breached, not one pillar maximised.** Option A is the magnificent bridge: it beats B on nothing Meera measures and loses on two pillars. That asymmetry, capable but misaligned, is how 1.6 writes its best distractors.

Trap: the unpriced option. When an option's cells cannot be filled with numbers at all (improves experience, future-proof), it is not a candidate. It is a wish. An option you cannot score against the stated targets loses to any option you can, even a humbler one.

Domain 3 connection

The feedback stage is what the lab's E7 cameras instrument: structured log lines are the raw material of next week's eval set. The SLA pillar's number is the p95 the digest printed. The cost pillar is the 47 paise on the One Ticket bill. Domain 1 names the pillars; Domain 3 is where their numbers come from.

***The one thing to remember.** Four stages, and the missing one is usually feedback or the human gate. Five pillars, each with a number, and the winning option is the one that breaches none, not the one that maximises one.*

The Atlas, the Blueprint Book and the Memory Map

Part Two, Part Three and the closing pages of the brief. The seven pictures, how to sit the forty questions, every answer's tell, and the twenty-five things to carry into the room.

The Architecture Atlas, A1 to A7

The Atlas draws what the stems describe in words. The brief's instruction for Day C is to redraw A4 from memory. Here is what each picture is for:

Atlas	What it draws	How to use it in a stem
A1 Taxonomy tree	Three floors, seven leaves: augmented LLM; five workflows; the agent loop	If the stem's answer is not one of the seven leaves, re-read the stem. Match its phrase (predefined code paths versus directs its own process) to the floor, then pick the leaf
A2 Five workflow shapes	Chaining as a line with gates; routing as a fan to wards; parallel as fan-out to an aggregate in code; orchestrator-workers as a planner over workers into synthesis; evaluator-optimizer as a rubric loop with a budget and a PASS exit	Recognise a diagram described in prose. Learn the shapes, not the paragraphs
A3 Reference architecture	Four bands: input, processing, output, feedback, with the human gate on refunds and the purple feedback arrow	Count the four stages against every 1.2 option. The missing arc is the answer
A4 Pattern selector	A walk of yes/no diamonds from one question to the agent loop	The 1.3 decision as a path, not a memory feat. Walk it for every pattern stem
A5 Topologies	Single agent, orchestrator plus subagents with dotted field offices, peer handoff with a baton	The dots are the point: notes stay inside, only reports travel. Quote the caps: depth 3, concurrency 20, max_budget_usd
A6 Decomposition tree	KF-2481's run: planned boxes solid, the dropped one crossed out, the grown ones dashed	Static versus dynamic in one glance. Dashed boxes are evidence the dynamic machinery earned its keep
A7 Value-pillar matrix	Three options, five pillars, every cell a number	No-red-column beats tallest-single-bar. An unpriced option is a wish

The A4 walk, in words

Ask the questions in order and stop at the first yes.

- One question, no multi-step work at all? Yes: **augmented LLM** (E1).
- Subtasks fixed and known in advance? Yes: known categories means **routing** (E3); a sequence means **chaining** (E2); independent subtasks under a latency budget means **parallel** (E4).
- Plan can be written once, then executed? Yes: **orchestrator-workers** (E5).
- Findings can invalidate the plan mid-run? Yes: **re-planning with a growth cap** (E8).
- Path itself unknowable, each result steers the next act? **Agent loop** (E7) with budget, exit and escalate.

Overlay questions asked at every node: a rubric exists and drafts improve, add the evaluator (E6). High stakes and 3x cost is acceptable, add voting. Money, law or people, add a human gate (Domain 5).

The Blueprint Book: how to sit it

Forty questions, six per objective and eight each for 1.3 and 1.5. Twelve are head-to-head pairs where two options look right; eight carry a number that decides. Rules: underline the deciding clause before reading the options; mark, do not dwell, about 110 seconds each; keep the key sealed until question 40; afterwards give every miss a ledger row with a family code. On track means 30 or more with no objective below 4 of 6 or 5 of 8.

The key, one tell per question

The book's key gives the tell, the reason and a family tag for each wrong option. Condensed here so the pattern across forty questions is visible. Read the tell column as a list of phrases to underline.

Q	Ans	The tell, in a few words	Obj
1	B	Deploy AI is ambition; the note names no workload, constraint or measure	1.1
2	C	Already fully deterministic: deterministic logic stays in code	1.1
3	B	40 ms, hard control loop, structured data: three disqualifiers	1.1
4	B	38 percent need information not in the ticket: draft plus human completion	1.1
5	C	Only one option names workload, categories, numbers and a gate	1.1
6	B	Measurable value in 4 weeks: volume times safety beats drama	1.1
7	B	The flow ends at output: the feedback arc is missing	1.2

Q	Ans	The tell, in a few words	Obj
8	B	Scrub PII before any model call; after is too late by definition	1.2
9	B	Irreversible plus money defines the gate; 97 percent still mis-approves 3 in 100	1.2
10	B	Three pipelines times three prompts: normalise at input, one pipeline	1.2
11	B	Feedback measures what happened after output met the world: edit distance, re-open rate	1.2
12	B	Cannot classify at all asks for an exit: a human queue with the ticket intact	1.2
13	B	Identical steps every time, accuracy over speed: chaining, verbatim	1.3
14	C	The sequence cannot be scripted: the agent, with a budget and escalation	1.3
15	A	while True and an else that swallows: both organs missing	1.3
16	B	max_tokens means the page ran out, not the thought: append, Continue, stay in budget	1.3
17	A	tool_use, tool_use, end_turn: fuel, fuel, natural exit	1.3
18	B	Six known templates, 30,000 a day, 2 seconds: all three shout router	1.3
19	A	Must never reach is a guarantee; only code between steps gives one	1.3
20	A	No multi-step anything: the cell suffices	1.3
21	B	40 documents, tight context: isolation keeps the reading out of the parent	1.4
22	B	Never by instruction is persuasion; by tool-list omission it is physics	1.4
23	B	A fresh subagent inherits nothing; the brief must carry ids, findings, constraints	1.4
24	B	Independent plus 40 s against 3 times 30 s: concurrency turns a sum into a max	1.4
25	B	Clean sequential break, no synthesis: a baton, not a hub	1.4
26	B	A coordinator over the coordinator: governance is what caps are for	1.4
27	B	Subtasks listed in the stem itself: the plan pre-exists	1.5
28	B	Cannot be predicted: the one phrase that unlocks the foreman	1.5
29	B	Re-planning is a loop; every loop needs a budget	1.5
30	B	A finding invalidated part of the plan: drop, add, count against the cap	1.5
31	B	Does the remaining plan still hold is the smallest question that decides	1.5
32	B	Escalating uncertainty, escalating machinery: the three-tier ladder	1.5
33	B	needs is a design statement: data flows along declared dependencies only	1.5
34	B	400 independent plus one synthesised report: fan-out with a built-in synthesise	1.5
35	B	Pillar plus number plus measurement mechanism	1.6
36	B	Rs 7.10 against a Rs 6 ceiling: one number decides	1.6
37	B	New service line from patterns is the transformation pillar's definition	1.6
38	B	A beats B on nothing Meera measures: the matrix already answered	1.6
39	B	A gain is a difference; one of the two numbers must precede deployment	1.6
40	B	Board time buys pillars and numbers, never mechanism names	1.6

Two things to notice. The shaded rows are 1.3 and 1.5, the soft spots, and nearly every tell in them is a quoted dividing line: identical steps, cannot be scripted, cannot be predicted, does the plan still hold. Second, the distractor families in the key are the same ones Domain 3 used: OE, VG, TR, CM, HL, SF, SD. The Blueprint key tags one on every wrong option.

The Memory Map: twenty-five things, condensed

Group	Carry into the room
The tree and the laws	Augmented LLM, five workflows, agents. Workflows: predefined code paths. Agents: model directs its own process. Law 1 constraint picks the pattern. Law 2 workflows when the path is known. Law 3 budget plus exit (E6 max_rounds and PASS; E7 BUDGET and end_turn; E8 max_new). Law 4 decompose on discovery. Law 5 a pillar with a number; alignment is no pillar breached.
The five workflows	Chaining: fixed steps plus gates. Routing: cheap classifier, specialist ward and model per category. Parallel: sectioning is coverage, wall clock equals the slowest scout; voting is confidence at 3x cost, for judgement not facts. Orchestrator-workers: model writes the subtask list, code steers delegate, work, synthesise. Evaluator-optimizer: needs a real rubric; criticism and draft go back together.
The loop, your 50 percent row	Seven stop_reason values, all HTTP 200. tool_use is fuel: append the assistant turn, run the tools, tool_result by tool_use_id, the model's reply decides the next turn. max_tokens is truncated not done. refusal is a different path, no blind retry. Wildcard escalates. The floor: budget spent, case open, human hand-off with a reason. Healthy trace: tool_use, tool_use, end_turn. The trap: an else that swallows.
Decomposition, your 75 percent row	The ladder: knowable to chain in code; plan writable once to E5; findings can invalidate to E8 re-plan with a growth cap. Subtasks listed in the stem means never orchestrator-workers. The replan call sees only finding plus remaining. Data flows along declared needs only; for context without a dependency is noise.

Group	Carry into the room
Multi-agent and the boardroom	Subagent starts fresh: brief carries ids, findings, authorization state. Isolation: notes stay in, reports return. Never do X: omit the tool. Caps not committees: depth 3, concurrency 20, max_budget_usd. Pillars: efficiency, cost, productivity, SLA, transformation; the stated pillar is the contract. Board sentence: 5 minutes not 4 hours, Rs 4 each, refunds above Rs 10k still cross a human desk.

The miss ledger

One row per miss: date, source, question, objective, family, the tell you skipped in your own words, fixed or not. The brief's pattern rule: three misses sharing a family beats three misses sharing an objective. Fix the habit, not the topic. Three CM rows means slow down and underline before the options. Three OE rows means re-walk A4 before the next sitting.

Domain 1 and Domain 3, side by side

Domain 1 idea	Where you already ran it in Domain 3
Routing workflow (E3): classifier picks a specialist	e6_shape_router.py: regex picks lookup, aggregate or RAG
Tool schema as a promise (E1); the phone book	e1_kaveri_mcp.py served it; e2_bloat_audit.py priced it
Tool restriction: an omitted tool does not exist	e3_gate_pass.py scopes; the Wrong Tower's prompt-based permissions flaw
Agent loop budget and timeout	One Ticket's 3 second carrier timeout and the 4 second SLA
Feedback stage, outcomes logged	e7_dock_cameras.py JSON lines and the p95 digest
Cost pillar, Rs 6 ceiling	e8_two_bills.py and the 47 paise bill
Context isolation	the 5,800-token window versus the 362k monolith
Parallel sectioning	carrier ETA running concurrently with retrieval's tail

The one thing to remember. Seven leaves on the tree, a walk through A4 for every pattern stem, two organs in every loop, four stages in every architecture, a number on every pillar. The Blueprint key tags every wrong option with a family. Ledger the family, not the topic.